

Tutorato Programmazione 2

Michele Porcellato, Giacomo Vettore

27 Maggio 2026

1 Note

In caso di dubbi, consultare la *documentazione Java*. Sarà accessibile anche durante l'esame.

2 Esame

Si progetti **Bro-Stelle**, un'applicazione che permette di usare diversi tipi di *Bros*. Un Bro ha un nome (stringa), un livello (da 1 a 11), un tipo (che determina la sua vita minima e massima e la sua velocità e che indica un colore per la GUI), e un attacco. I diversi tipi di Bros sono:

- **Tank** (MAGENTA): vita minima: 8000, vita massima 10500, velocità: lenta;
- **Long-ranged**: velocità: normale; e si dividono in:
 - **Sniper** (OLIVE): vita minima: 3000, vita massima 5500;
 - **Artillery** (CYAN): vita minima: 2000, vita massima 3500;

Un Bro di livello 1 ha la vita minima dettata dal suo tipo. Per ogni livello successivo all'1, il Bro aggiunge alla sua vita il 10% della vita minima del suo tipo, fino a un massimo uguale alla vita massima del tipo. Per esempio, un Tank di livello 1 ha 8000 di vita, a livello 2 ha 8800 di vita, a livello 3 ha 9600 di vita, ma a livello 11 ha 10500 di vita.

Un attacco ha un nome (stringa), un danno base (`int`), un tipo (che determina la portata e la traiettoria). I diversi tipi di attacco sono i seguenti:

- **Single**: portata: molto lunga (150px), traiettoria: terreno;
- **AoE**: portata: media (100px); e si dividono in:
 - **Lobber**: traiettoria: aerea;
 - **Blast**: traiettoria: terreno;

Il danno che fa un attacco è il danno base più il 10% per ogni livello del Bro (senza limite, a differenza della vita del Bro).

L'applicazione è inizializzata con questi Bros:

- **Franco** (di tipo Tank), lv 2, con attacco: **SBAM** (di tipo Blast), 1000 danni base;
- **Dinamichele** (di tipo Artillery), lv 8, con attacco: **Bomba** (di tipo Lobber), 2000 danni base;
- **Ape** (di tipo Sniper), lv 11, con attacco: **Bzz** (di tipo Singolo), 2000 danni base;
- **Edoardo** (di tipo Tank), lv 1, con attacco: **Sciarpa** (di tipo Singolo), 500 danni base;
- **Salice** (di tipo Sniper), lv 1, con attacco: **Tentacolo** (di tipo Lobber), 1200 danni base;

Come si vede in figura, l'applicazione mostra: in alto una lista di Bros; al centro uno spazio **BLACK** di 300x300 px, dove viene mostrato il Bro selezionato; in basso un obiettivo da attaccare; a sinistra un bottone per ordinare i Bros per vita e uno per ordinarli per nome.

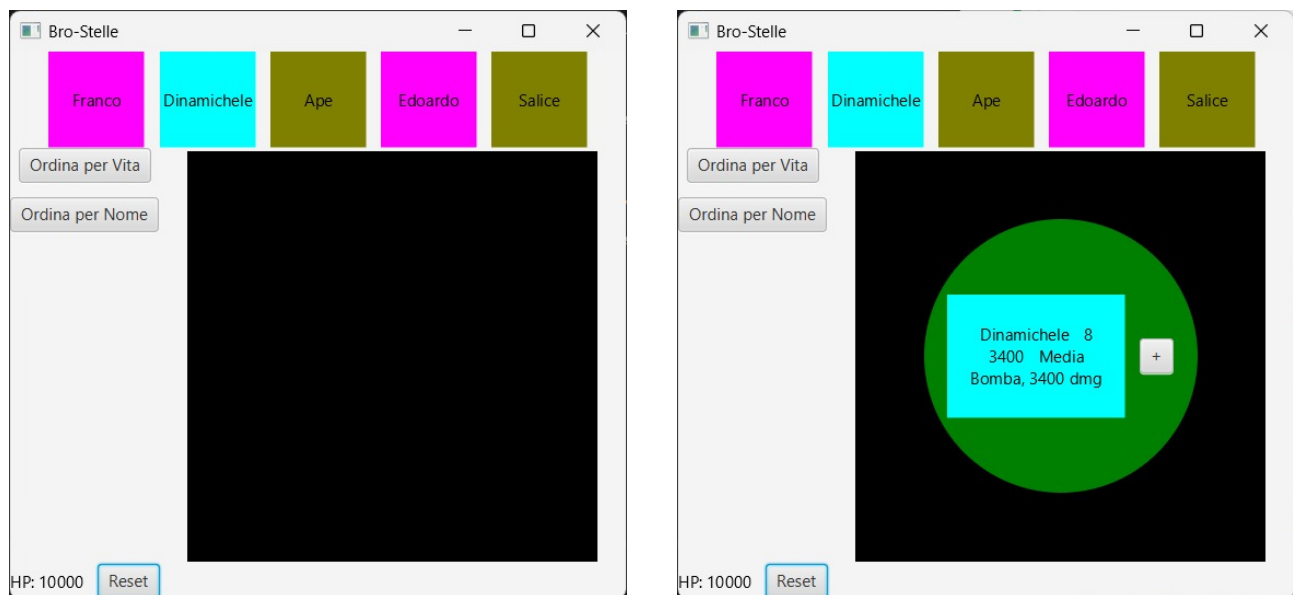
Nella lista, i Bros sono mostrati come un quadrato del colore del tipo del Bro, di 70px con al centro il nome del Bro. L'obiettivo da attaccare si presenta come un testo indicante il totale di HP (inizializzato a 10.000) e un bottone che resetta gli HP a 10.000.

Una volta cliccato un Bro dalla lista in alto, viene visualizzato al centro dell'applicazione, affiancato da un bottone "+" e circondato da un'area **GREEN** che indica la sua portata di attacco. Al centro, i Bros sono mostrati come un rettangolo del colore del tipo del Bro, di 130x90px, con al centro il nome, il livello, la vita, la velocità del Bro, seguiti dall'attacco del Bro. L'attacco del Bro indica il nome e il danno.

Premere "+" aumenta il livello del Bro di 1, a meno che non sia a livello 11, in qual caso si mostra un messaggio di **ALERT** di tipo **ERROR** che dice "Livello massimo raggiunto". **Nota:** quando aumenta il livello, cambiano vita e danno dell'attacco, e queste informazioni vanno aggiornate quando si preme "+".

Cliccare nell'area **GREEN** del Bro emette un attacco che colpisce l'obiettivo e ne aggiorna gli HP. La grandezza dell'area dipende dalla portata dell'attacco, come indicato nella lista degli attacchi. Quando il bersaglio viene ucciso, si mostri un messaggio di **ALERT** di tipo **ERROR** con un messaggio appropriato e si resetti il bersaglio.

L'ordinamento naturale dei Bros è per nome, alfabetico, inoltre si possono ordinare per vita (crescente); in caso di conflitti si ordini usando l'ordinamento naturale.



- Si usi, quando evidente/utile, una gerarchia di ereditarietà.
- Si usino costanti ove ragionevole, e si badi alla pulizia del codice, evitando duplicazioni e codice inutilmente complesso.
- La disposizione degli elementi grafici deve rispecchiare quanto mostrato; tuttavia, ciò è considerato meno importante rispetto alla corretta funzionalità dell'interfaccia e del resto dell'applicazione.
- Si richiede il **class diagram UML**. È sufficiente la vista di alto livello (no attributi/metodi) ma devono essere mostrate le associazioni più importanti, oltre a quella di ereditarietà. Il diagramma UML deve essere consegnato su un foglio di protocollo su cui sono indicati nome, cognome, numero di matricola.
- Ricordatevi delle **ConcurrentModificationException** nelle collection di JavaFX (quindi occhio a usare il costrutto **foreach**).